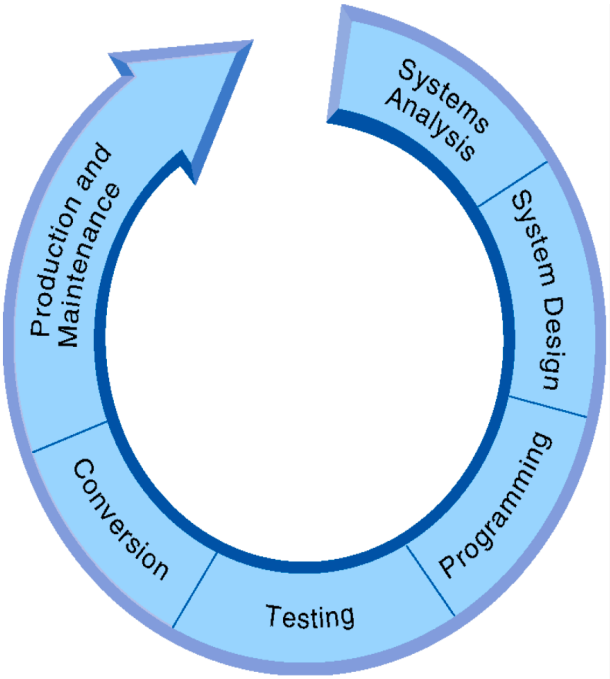
Lecture 2

(Software Development)

❖ Life Cycle (SDLC)



1. **System Analysis:** analysis of business problem (includes feasibility study)
   - Defining the **problem** and **identifying causes**
   - Specifying **solutions** (write report describes costs and benefits of each)
   - Identifying **information requirements** to be met

2. **System Design:**
   - Describe **system specifications** that will deliver functions identified during systems analysis
   - Should address all **managerial**, **organizational**, and **technological** components of system solution

3. **Programming:**
   - **specifications from design stage** are translated into **software program code**
   - **Software** may be **purchased**, **leased (subscription)**, or **outsourced.**

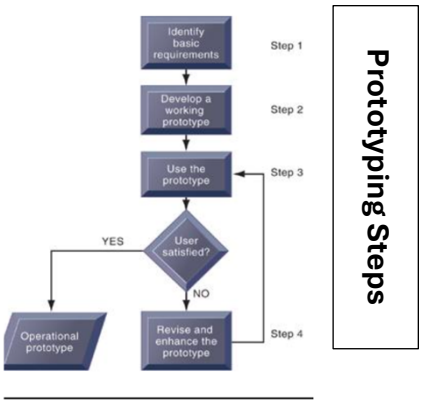
4. **Testing:** to ensure system produces right results
   - **Unit testing:** Tests **each program** in system **separately**
   - **System testing:** Tests **functioning** of **system as a whole**
   - **User Acceptance testing (UAT):** Makes sure system **is ready** to be used in production

5. **Conversion:** changing from **old system** **to** **new system**
   - **Finalization of detailed documentation** showing **how system works from technical and end-user standpoint (**Requires **end-user training)**
   - Four main strategies:
     1) Parallel Strategy
     2) Direct cutoff
     3) Pilot study
     4) Phased approach

6. **Production & Maintenance:**
   to correct **errors**, meet **new requirements**, or improve processing **efficiency**

❖ **System Building Approaches**

1. **Traditional Approach (Waterfall)**
   - **Phased approach** - divides development into **formal stages**
   - Emphasizes **formal specifications** and **paperwork**
   - **Still used** for building large complex systems that have known
   - Can be **costly, time-consuming,** and **inflexible**
2. **Prototyping (Agile)**
   - **Advantages:**
     1- Useful if some **uncertainty in requirements** or design solutions
     2- Often **used for end-user interface design**
     3- More likely to **fulfill end-user requirements**

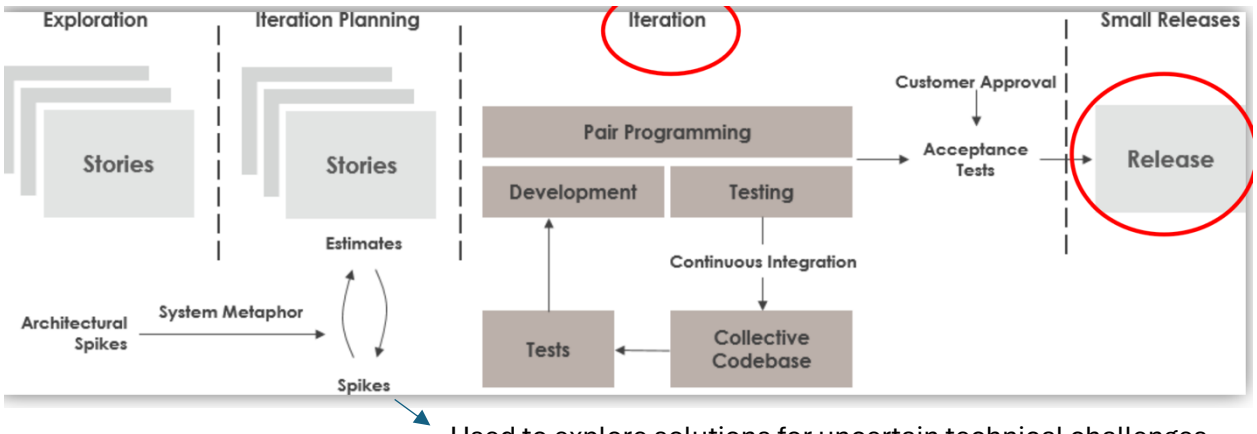


   - **Disadvantages:**
     1- **May not accommodate large quantities** of **data or large number of users**
     2- May **not undergo full testing** or documentation
   - **Agile:** a **method based on iterative and incremental development**
     - **4 Core Values**
       1) **Individuals and interactions** over processes and tools
       2) **Working software** over comprehensive documentation
       3) **Customer collaboration** over contract negotiation
       4) **Responding to change** over following a plan

     - **Different Frameworks:**
       1- **Scrum** (Sprint-Based Approach)
         a. **Roles**
            i. **Scrum Master:** Facilitates the process, removes obstacles.
            ii. **Product Owner:** Defines vision, prioritizes backlog.
            iii. **Development Team:** Implements, delivers increments.
         b. **Events**
            i. **Sprint:** 2-4 week development cycle.
            ii. **Daily stand-ups:** Quick progress check-in.
            iii. **Sprint Review:** Showcases completed work.
            iv. **Sprint Retrospective:** Evaluates & improves process.
         c. **Artifacts**
            i. **Product Backlog:** Prioritized task list.
            ii. **Sprint Backlog:** Tasks for the sprint.
            iii. **Increment:** Deliverable product update.

2- **Extreme Programming** (XP- Faster Sprints, more testing)

a. Continuous testing and quick improvements



Used to explore solutions for uncertain technical challenges.

3- **Kanban** (No Sprints, Continuous Flow)

a. **does not prescribe specific roles or time-boxed iterations (sprints)**

b. uses a **board with columns (To Do, In Progress, Done)**

3. <u>**End-user Development**</u>

- users build their own systems
- Uses **4GL tools** (easy, low-code/no-code)
  - PC tools (Word, Access)
  - Query languages (SQL)
  - Report tools
  - Graphics tools
  - App generators

4. <u>**Application**</u>

- **Software Package**
  - **pre-built programs** that companies can buy and use directly instead of building from scratch.
  - **Advantages: saves time and money, reliable, customizable**

- **Outsourcing**
  - **hiring an external company (vendor)** to handle your IT needs instead of doing it internally.
  -

- **Advantages:**

  - **Flexibility** → scale up/down easily حسب احتياجك
  - **Cost efficiency (Economies of scale)** → vendors serve many clients, so cost is lower
  - **Focus on core business** → company focuses on what it does best (e.g., sales), not IT

- **Disadvantages**
  - **Hidden costs** → unexpected fees (maintenance, upgrades, communication)
  - **Loss of control** → you depend on external company
  - **Offshore risks** → time zones, communication issues, quality differences

- **Types of outsourcing**

  1- ASP / SaaS (Application Service Provider)

  - ➢ You **don't own the software**, You **use it (online)**, The provider handles everything (servers, updates, maintenance)

  2- External Vendors (Domestic or Offshore)

  - ➢ **You hire the company (locally or from another country) and they build the software for you**

Answers from ChatGPT

Case	Approach
1	Waterfall
2	Agile (Scrum)
3	Agile (XP)
4a	Agile (Kanban)
4b	Outsourcing (Fixed-price)
4c	Waterfall
4d	External Vendor (Cost-plus)